DA IDEIA À SOLUÇÃO

Crie Sistemas com IA e Ganhe Dinheiro



Do zero ao sistema no ar:
aprenda a resolver problemas reais
e vender soluções, mesmo começando
sem saber programar.

Warley Ferraz Almeida

1ª EDIÇÃO

BIBLIOTHEON

AMOSTRA GRATUITA

Comece por uma necessidade real

Seleção da 1ª edição: apresentação, fundamentos, ambiente, primeiro uso da IA e método profissional. O livro completo segue até construção, testes, VPS, domínio, operação e proposta comercial. Os apêndices incluem código e modelos. Esta amostra não é o livro completo.

Crie Sistemas com IA e Ganhe Dinheiro

Do zero ao sistema no ar: aprenda a resolver problemas reais e vender soluções, mesmo começando sem saber programar.

Você percebe uma tarefa repetitiva, uma informação perdida entre mensagens ou uma planilha que já não acompanha o trabalho. Consegue imaginar uma solução, mas não sabe programar. Este livro começa nesse ponto: transformar uma necessidade concreta em um sistema que você compreende, testa e coloca no ar.

A inteligência artificial ajuda a escrever código e explicar erros. O seu papel é conduzir: dizer o que o sistema precisa fazer, reconhecer se a resposta atende ao problema e conferir o resultado. Você aprenderá o básico necessário enquanto constrói, sem precisar dominar uma linguagem inteira antes de começar.

O método nasce da prática profissional de Warley Ferraz Almeida. O autor usa esse processo com frequência para desenvolver sistemas e já recebeu dinheiro por trabalhos realizados com ele. A organização didática, as explicações e os procedimentos complementares foram desenvolvidos com pesquisa e assistência de IA. Não há valores de faturamento, depoimentos ou casos de clientes apresentados como prova de resultado típico. Ganhar dinheiro depende de resolver uma necessidade, negociar e entregar um serviço útil; a leitura não garante contratação ou renda.

O que você vai construir

Vamos acompanhar a Oficina Clara, uma empresa fictícia que precisa organizar ordens de serviço. O gerente cria e acompanha atendimentos; cada técnico vê somente os seus e avança o trabalho

até a conclusão. Esse exemplo pequeno permite aprender requisitos, contas, permissões, banco, interface, testes e operação sem trocar de negócio a cada página.

Você construirá por etapas numa pasta própria e terá uma implementação de referência para estudar e comparar. Depois seguirá o percurso de uma VPS Ubuntu com PostgreSQL, domínio, HTTPS, recuperação de acesso, cópia e atualização. Finalmente, aprenderá a adaptar o método e preparar uma proposta para um cliente. O escopo é uma aplicação web pequena; sistemas clínicos, financeiros críticos, de alta escala ou com operação offline exigem conhecimentos e revisão adicionais.

Como percorrer o livro

Leia os fundamentos e execute as oficinas na ordem. Cada etapa termina com algo observável: uma regra escrita, um programa que inicia, uma operação recusada ou uma recuperação conferida. Quando aparecer um erro, registre onde aconteceu antes de pedir uma correção. Aprender a diagnosticar faz parte do método.

Use Windows 11 com Ubuntu no WSL, Ubuntu ou macOS. Os comandos principais usam um terminal compatível com Unix; a preparação de cada sistema é explicada. A VPS de destino usa Ubuntu independentemente do seu computador. Nos exemplos, nomes como `SEU-DOMINIO` são substituições intencionais, explicadas junto do comando; nunca são credenciais prontas.

O livro inclui código e modelos essenciais nos apêndices, permitindo reconstruir a referência mesmo em lojas que entregam apenas a leitura. Na oferta de pacote, projeto, caderno, exemplos, prompts e checklists também acompanham arquivos separados. Use os capítulos como percurso principal e os apêndices como consulta. Para copiar código, prefira o EPUB ou os arquivos separados. No PDF, confira se a cópia inseriu quebras dentro de nomes ou textos longos; a quebra visual de página não deve alterar o programa.

Custos e preparação

Você precisa de um computador autorizado, internet e tempo para praticar. O laboratório local não exige contratar VPS. Python, Django, Git e VS Code podem ser usados sem comprar o livro de outro fornecedor; suas licenças e condições continuam aplicáveis. O assistente de IA pode exigir assinatura ou cobrança por uso.

Hospedagem, registro e renovação de domínio, envio de e-mail, armazenamento de backup e eventual revisão especializada são despesas separadas. Antes de contratar, compare preço inicial, renovação, limites, impostos e cancelamento. Os exemplos comerciais são didáticos e não representam cotação de fornecedores.

Créditos e condições desta edição

Warley Ferraz Almeida · Bibliotheon. **1ª edição · Português brasileiro · Outubro de 2026.** Pesquisa, organização, redação e revisão editorial assistidas por IA, sob responsabilidade do autor. Não se declara revisão técnica independente ou teste com compradores.

Você pode estudar e adaptar os modelos e o código original em projetos próprios, inclusive comerciais, respeitando as licenças das dependências. Essa permissão não autoriza revender ou redistribuir o livro, sua capa ou seus textos como produto próprio. As marcas citadas pertencem aos respectivos titulares; não há vínculo comercial declarado com os fornecedores.

Atendimento sobre compra, acesso e integridade dos arquivos: utilize o canal indicado no pedido e na plataforma em que comprou. Programação individual, implantação para o leitor, consultoria e mentoria não estão incluídas. Esta compra corresponde à 1ª edição; não inclui compromisso de novas edições ou atualizações futuras.

1. Antes de começar: você não precisa saber os nomes

Clara tem uma oficina e um problema comum: as informações chegam por mensagens, algumas ficam num caderno e outras na memória. Quando um cliente pergunta pelo serviço, ela procura em três lugares. Você pode ter uma realidade diferente: uma associação que empresta equipamentos, um pequeno negócio que agenda atendimentos ou um projeto pessoal. O ponto de partida é parecido. Existe uma atividade, existe uma dificuldade e você quer organizar melhor o trabalho.

Neste livro, Clara, Ana e Bruno são personagens fictícios. Vamos acompanhá-los para não precisar aprender um vocabulário e um negócio novos a cada capítulo. Clara gerencia; Ana e Bruno executam serviços. Quando a necessidade ficar clara, pediremos à IA que a transforme em arquivos, regras e telas. Depois conferiremos o resultado como pessoas que precisam trabalhar com ele.

A novidade não é apertar um botão que transforma desejo em produto perfeito. É conseguir conversar com uma ferramenta capaz de produzir código enquanto você aprende a dirigir o processo. Você não precisa decorar uma linguagem inteira para começar. Precisa aprender a pedir, observar, comparar e reconhecer quando falta informação. Este livro ensina essas ações junto com a parte técnica.

O computador, a mesa e a oficina

Pense no computador como uma oficina de trabalho. O **sistema operacional** organiza a oficina: permite abrir programas, guardar arquivos, conectar à internet e usar o teclado. Windows, macOS e distribuições Linux cumprem esse papel. Ubuntu é uma

distribuição Linux: um conjunto organizado de sistema e programas, com sua própria forma de instalar e atualizar coisas. Não é um site nem uma marca de computador.

Um **programa** é uma ferramenta que funciona dentro dessa oficina. O navegador abre páginas. O VS Code permite organizar e editar os arquivos de um projeto. O terminal recebe instruções escritas. Você pode ter os três abertos ao mesmo tempo, cada um mostrando uma parte diferente do trabalho.

Um **arquivo** é uma unidade de conteúdo com um nome. Uma **pasta** reúne arquivos e outras pastas. O nome `ideia.md` significa que o arquivo usa a convenção Markdown; não significa que a ideia já virou sistema. Um **caminho** informa onde o arquivo está: `docs/ideia.md` significa arquivo `ideia.md` dentro da pasta `docs` do projeto. A **raiz do projeto** é a pasta principal que você abriu no editor. Não é necessariamente a raiz do seu disco.

Antes de qualquer instalação, pratique no gerenciador de arquivos do seu computador. Crie uma pasta chamada `meu-primeiro-sistema` dentro de uma área pessoal que você saiba encontrar. Abra essa pasta. Observe que ela está vazia. Volte uma pasta e abra novamente. Esse gesto simples tem importância: muitos erros atribuídos à IA são comandos executados no lugar errado.

Quem faz o quê no sistema que vamos construir

Imagine que Clara digita uma descrição e clica em Salvar. O navegador envia um pedido. A aplicação confere se Clara entrou com a conta correta e se pode criar aquela ordem. Depois guarda a informação no banco de dados e responde. O navegador apresenta a nova lista.

Interface é a parte com a qual a pessoa interage: textos, campos, botões, tabelas. **Servidor da aplicação** é o programa que recebe pedidos e aplica regras. **Banco de dados** é onde ficam registros organizados e relacionados. A mesma máquina pode executar aplicação e banco num projeto pequeno; isso não torna as duas coisas iguais.

A descrição da ordem pertence aos dados. A regra que impede Bruno de ver uma ordem de Ana pertence ao programa. Trocar uma cor da tela altera apresentação. Apagar o banco pode apagar as ordens. Essa distinção acompanhará nossas escolhas de cópia e atualização.

Um **data center** é uma instalação física onde ficam computadores, rede, energia e outros recursos de infraestrutura. Uma **VPS** é uma máquina virtual oferecida por um provedor, executada sobre infraestrutura física. “Nuvem” não significa que seus arquivos deixaram de existir num equipamento. **Database**, em inglês, significa banco de dados; não é o nome do prédio onde está a VPS.

Você usa seu computador para desenvolver. Quando quiser disponibilizar o sistema pela internet, usará uma máquina preparada para isso. No método do autor, uma VPS de desenvolvimento/demonstração recebe a versão de ensaio, e a VPS do cliente recebe a versão aprovada. Em um sistema próprio, a produção pode ficar na própria conta do autor, com separação de ambientes e acessos.

Três lugares que não devem se confundir

Lugar	Como reconhecer	O que fazer nele
Seu computador	Janela local do VS Code e terminal local	Criar arquivos, desenvolver e testar com dados fictícios
Repositório remoto	Página do projeto no GitHub	Guardar histórico de código e compartilhar acesso autorizado
VPS	Sessão remota identificada pelo usuário e servidor	Preparar e operar a instalação de demonstração ou produção

O GitHub não é o endereço onde os clientes vão usar nossa aplicação. Ele guarda o projeto. A VPS não contém automaticamente o que você acabou de editar no VS Code. E fechar o navegador não necessariamente para o servidor. Sempre pergunte: “Onde esta ação acontece?” antes de copiar um comando.

Aprender com pouca experiência, em qualquer idade

Idade não determina habilidade digital. Alguém de 60 anos pode ter anos de prática; alguém de 20 pode nunca ter criado uma pasta. Nosso ponto de partida é não saber programar e ter pouca familiaridade com ferramentas técnicas. O texto explica o necessário, mas não trata o leitor como incapaz.

Este livro foi organizado para adultos iniciantes. Menores podem estudar conceitos com acompanhamento, mas contas, serviços e contratações precisam respeitar os requisitos de idade e responsabilidade dos fornecedores. Não use informação falsa nem compartilhe credenciais para contornar termos.

Como estudar sem transformar leitura em maratona

Faça uma unidade de trabalho por vez, não um número arbitrário de páginas. Uma unidade termina quando existe algo que você consegue conferir: uma ideia escrita, uma tela que abre, uma regra testada. Você pode ler num ritmo e executar em outro. Anote onde parou; deixar uma dúvida registrada é progresso.

Cada oficina deste livro segue um movimento: entender a situação, preparar a entrada, fazer uma ação, observar a saída e experimentar uma pequena mudança. A seção de resposta comentada ajuda a conferir o raciocínio. Não existe certificado automático, e marcar uma caixa não substitui a execução.

Crie `diario.md` no projeto pelo VS Code, quando o instalar, ou escreva num caderno por enquanto. Use quatro linhas: “Hoje tentei”, “O que aconteceu”, “O que conferi” e “Próximo passo”. Evite escrever somente “não deu certo”. A descrição concreta torna possível pedir ajuda e retomar sem recomeçar.

Se o texto estiver pequeno, aumente o zoom do editor e do leitor. Prefira o EPUB quando quiser alterar a fonte; no PDF, use zoom e modo de leitura. Faça os exercícios com calma e mantenha só as janelas necessárias visíveis. A aprendizagem não precisa acompanhar a velocidade das respostas da IA.

Primeiro desafio, ainda sem programar

Você quer organizar empréstimos de ferramentas numa associação. Escreva quem entrega a ferramenta, quem recebe, o que precisa ser anotado e como saber se ela voltou. Depois escreva três coisas que a primeira versão não fará.

Resposta comentada: uma possibilidade é registrar ferramenta, pessoa autorizada, data de retirada e devolução. O responsável marca a devolução; um participante comum não altera o registro dos outros. Fora do escopo podem ficar pagamentos, reconhecimento facial e compra de peças. Há outras respostas válidas. O importante é conseguir verificar o resultado e dizer onde a solução termina.

Você pode avançar quando: encontrar sua pasta, distinguir programa de dados e explicar por que a VPS não é um banco de dados. Ainda não precisa saber instalar um servidor. Essa será uma habilidade construída depois que houver algo para hospedar.

2. Preparar Windows 11, Ubuntu ou macOS

Um arquivo guarda conteúdo; uma pasta agrupa arquivos. A extensão indica uma convenção de formato: `.md` é texto Markdown, `.py` é código Python, `.yaml` costuma guardar configuração. Salve arquivos como texto, sem deixá-los virar `contexto.md.txt`. No VS Code, o explorador lateral mostra o nome real.

Terminal é a interface em que você escreve comandos. O local atual importa: instalar dependências na pasta errada ou executar outro `manage.py` gera confusão. `pwd` mostra onde você está, `ls` lista arquivos, `cd nome` entra em uma pasta e `cd ..` volta um nível. Esses comandos funcionam no Ubuntu, WSL e terminal do macOS. Não copie o símbolo de prompt, caso apareça em uma fonte externa.

Trilha A — Windows 11 com Ubuntu no WSL

O WSL executa um ambiente Linux no Windows. Antes, salve trabalhos abertos e tenha permissão administrativa. No menu Iniciar, procure PowerShell, execute como administrador e rode:

```
wsl --install -d Ubuntu-24.04
```

Reinicie se solicitado. Abra Ubuntu pelo menu, aguarde a preparação e crie usuário e senha Linux. Ao digitar senha no terminal, pode não aparecer nenhum caractere; isso é esperado. O nome e a senha Linux não precisam ser os da Microsoft. Confira no PowerShell `wsl --list --verbose`; a distribuição deve usar versão 2. Se não usar, consulte o erro antes de converter e mantenha cópias de arquivos importantes. Virtualização e políticas de computador corporativo podem exigir apoio do administrador. Instalação oficial do WSL.

No Ubuntu recém-aberto pelo WSL, execute também o bloco de instalação de Python, venv e Git da trilha B abaixo. Ele prepara o laboratório dentro do Linux; não execute esses comandos no PowerShell.

Instale o VS Code pelo instalador oficial para Windows. Abra Extensões com Ctrl+Shift+X e instale WSL, publicado pela Microsoft. Guarde o projeto dentro do Ubuntu, por exemplo `~/projetos/oficina-clara`. No terminal Ubuntu, execute `code .` nessa pasta. O canto inferior da janela deve indicar WSL. Se `code` não existir, reinicie o terminal ou use a paleta do VS Code e "WSL: Connect to WSL". A documentação do VS Code explica essa integração.

Trilha B — Ubuntu

Use Ubuntu 24.04 LTS ou ambiente compatível, com acesso administrativo autorizado. No aplicativo Terminal:

```
sudo apt update
sudo apt install python3 python3-venv python3-pip git unzip
python3 --version
git --version
```

O comando `sudo` solicita permissão para alterar o sistema. Leia o que será instalado. Se houver falha de rede ou repositório, corrija a origem do erro; repetir dezenas de comandos diferentes torna o ambiente mais difícil de compreender.

Instale o VS Code pelo pacote `.deb` oficial correspondente à arquitetura, usando o instalador gráfico de pacotes. Abra-o pelo menu e escolha Arquivo → Abrir pasta. Em Ubuntu dentro do WSL, use o VS Code do Windows conforme a trilha A; não instale outro editor Linux por engano. Instalação oficial Linux.

Trilha C — macOS

Consulte “Sobre Este Mac” para distinguir Apple Silicon e Intel e conferir a versão do sistema. Baixe o VS Code oficial adequado, abra o arquivo baixado e mova o aplicativo para Aplicativos. Na paleta Cmd+Shift+P, procure “Shell Command: Install 'code' command in PATH”, caso queira abrir pastas pelo terminal. Instalação do VS Code no macOS.

Instale uma versão compatível de Python pelo instalador oficial macOS. Esta edição usa Django 5.2 e testa Python 3.14; prefira uma série suportada pelo conjunto de dependências, como 3.12–3.14, e não escolha automaticamente uma versão recém-lançada. Abra Terminal, em Aplicativos → Utilitários, e confira `python3 --version`. Para Git, execute `git --version`; se o sistema oferecer as ferramentas de linha de comando da Apple, conclua a instalação autorizada e repita a verificação. Não substitua o Python interno do sistema. Distribuições oficiais Python.

Onde guardar e como abrir o exemplo

Extraia o ZIP de complementos pelo gerenciador de arquivos. Localize `projeto-oficina`, que contém `manage.py`. Copie essa pasta para sua área de projetos. No VS Code, abra exatamente essa pasta; no menu Terminal → Novo Terminal, confira `pwd` e `ls`. No Windows, o terminal deve estar no Ubuntu/WSL, não no PowerShell. Use o explorador para copiar do Windows para a pasta Linux pelo acesso do WSL se necessário; confirme os arquivos no editor.

Os comandos abaixo são compartilhados pelas três trilhas, usando o terminal Linux ou macOS. O complemento `comandos-do-livro.md` guarda os blocos em texto editável, útil quando uma linha longa aparece quebrada visualmente no PDF. Substituições indicadas no texto continuam necessárias:

```
python3 -m venv .venv
source .venv/bin/activate
python --version
python -m pip install -r requirements.txt
```

O ambiente virtual `.venv` separa as bibliotecas desse projeto. Após ativá-lo, `python` e `pip` devem se referir a ele. `python -m pip` reduz a chance de instalar no Python errado. Se a ativação disser “arquivo não encontrado”, confira se o primeiro comando terminou sem erro e se você está na pasta correta. No Ubuntu, erro de `ensurepip` costuma indicar falta de `python3-venv`.

Editor, extensões e IA

O caminho completo com **Codex ou Claude Code pela extensão** está na oficina logo após este capítulo. Escolha uma ferramenta para iniciar; não precisa contratar as duas.

Instale Claude Code a partir da extensão oficial da Anthropic no VS Code, confira o publicador e entre em sua própria conta. No WSL, observe se a extensão pede instalação no ambiente remoto. Se o painel não aparecer, use a paleta e procure “Claude Code”. A extensão tem requisitos de conta e versão; confira-os na documentação oficial, sem assumir que a gratuidade de outro produto se aplica aqui. Não cole senha, token de cliente ou chave privada no chat.

Docker vem depois

Não é preciso Docker para o primeiro laboratório local. Quando chegar à infraestrutura, Ubuntu usa Docker Engine e plugin Compose; Windows e macOS podem usar Docker Desktop compatível com seu sistema/arquitetura. No Windows, habilite integração com a distribuição WSL nas configurações do Desktop. No Mac, abra o aplicativo e aguarde o motor iniciar. Confira `docker version` e `docker compose version` no terminal do projeto. Se o cliente existir e o servidor não responder, o motor pode estar parado ou sem permissão. Não resolva concedendo acesso mundial ao socket.

Consulte os requisitos e licença atuais antes de instalar: Windows, macOS e Ubuntu. Essas instalações não foram reproduzidas em Windows/macOS nesta edição.

Conferência: pare até conseguir localizar `manage.py`, ativar `.venv` e instalar as dependências sem erro. Registre sistema e versões no modelo de progresso. Isso evita pedir à IA que diagnostique um ambiente que nem você consegue descrever.

Visita guiada ao VS Code

Na janela do VS Code, o painel de arquivos costuma ficar à esquerda. Ele representa a pasta aberta, não todos os arquivos do computador. A área central mostra o arquivo selecionado. A barra inferior traz informações do ambiente. O terminal pode aparecer abaixo da área central. O painel da IA pode ocupar outra lateral. Mudar o tamanho de um painel não altera os arquivos.

Para criar um arquivo, primeiro abra a pasta certa em Arquivo → Abrir Pasta. No explorador do editor, use Novo Arquivo e digite o nome completo, por exemplo `ideia.md`. Escreva uma frase e salve com Ctrl+S; no Mac, Cmd+S. Feche a aba e reabra pelo explorador. Se a frase reapareceu, você verificou que foi salva. Uma conversa no painel da IA não é automaticamente esse arquivo.

Para criar `docs/contexto.md`, crie a pasta `docs`, depois o arquivo dentro dela. Markdown continua sendo texto: `#` inicia título, uma linha vazia separa parágrafos e `-` pode iniciar uma lista. Você não precisa aprender todas as opções para documentar o projeto. O arquivo deve ser compreensível mesmo sem formatação bonita.

A paleta de comandos procura ações do editor. Abra com Ctrl+Shift+P ou Cmd+Shift+P no Mac, digite parte do nome da ação e leia as opções. Ela não é o terminal nem o chat. Se você colar um comando de instalação na paleta, ele não será executado como no terminal. Quando uma instrução disser “no painel da IA”, procure o campo de conversa da extensão escolhida.

Um pequeno passeio no terminal

Abra Terminal → Novo Terminal. No ambiente Linux/WSL/macOS usado pelo livro, digite `pwd` e pressione Enter. A resposta é o caminho da pasta atual. Digite `ls` e veja os arquivos. Se abriu `meu-primeiro-sistema`, deve reconhecer `ideia.md` e `docs` quando já criados.

Agora digite `cd docs`, depois `pwd`. Você entrou na pasta de documentação. Digite `cd ..` e confira de novo. Os dois pontos significam a pasta acima. Isso não apaga nada. A habilidade que queremos é localizar onde uma instrução vai agir, não decorar comandos.

Comandos diferenciam letras e símbolos. `python --version` tem dois hífens normais. Aspas curvas copiadas de um texto formatado podem falhar. Os blocos deste livro e o arquivo de comandos dos complementos usam caracteres apropriados; se algo parece estranho após copiar do PDF, use a versão Markdown editável. Não copie os delimitadores do bloco nem palavras indicadas como substituição.

Quando um comando termina, normalmente surge outra linha pronta para receber entrada. Um servidor em funcionamento pode ocupar o terminal por tempo indefinido: isso é esperado. Ctrl+C pede interrupção do processo em primeiro plano. Não é o mesmo que apagar um arquivo e não deve ser usado no meio de uma migração de banco sem entender a consequência.

Windows: duas oficinas na mesma máquina

No percurso Windows adotamos WSL com Ubuntu para unificar os comandos do laboratório e aproximar o ambiente da VPS. Não afirmamos que seja a única forma de usar as extensões no Windows. Há suporte nativo que evolui; nossa escolha é pedagógica. Se você usa Windows 11, prepare o WSL conforme a trilha A antes dos comandos Linux.

No PowerShell, `wsl --list --verbose` mostra distribuições instaladas. Dentro do Ubuntu, `pwd` costuma começar em `/home/SEU-USUARIO`. Se você vê `C:\...` no prompt, provavelmente está num terminal Windows. Não cole `sudo apt` nele esperando que vire Linux. Abra Ubuntu pelo menu Iniciar ou conecte o VS Code ao WSL.

Para criar sua pasta no Ubuntu, execute ali `mkdir -p ~/projetos/meu-primeiro-sistema`, depois `cd ~/projetos/meu-primeiro-sistema` e `code .`. O ponto significa "esta pasta". Confira o indicador WSL na janela. Para levar os complementos baixados para o Linux, abra o Explorador de Arquivos do Windows, localize Linux/Ubuntu e sua pasta pessoal; copie o projeto para `projetos`. Confirme no VS Code que `manage.py` está na pasta aberta. Mantenha apenas uma cópia de trabalho ativa para não editar uma e executar outra.

Se a instalação falhar, registre a mensagem exata e a etapa. Em computador da escola ou empresa, peça ao administrador as permissões necessárias. Não desative proteções ou altere virtualização por tentativa aleatória. Você pode continuar escrevendo requisitos enquanto resolve o ambiente.

macOS e Ubuntu: o que muda e o que continua igual

No macOS, o Terminal e o VS Code são aplicativos diferentes. Você pode usar o terminal integrado do editor para manter a pasta correta. Um Mac com processador Apple Silicon usa arquitetura diferente de muitos servidores; por isso, construir uma imagem local não comprova que ela rodará no servidor sem atenção à arquitetura. No caminho deste livro, o build de implantação acontece no destino ou no ambiente compatível definido.

No Ubuntu instalado diretamente no computador, não existe a camada WSL. Você instala o editor Linux e abre sua pasta pessoal. Se usa outra distribuição Linux, os conceitos continuam, mas `apt` pode não ser o gerenciador de pacotes. A trilha reproduzível do texto é Ubuntu; não aplique comandos de distribuição diferente sem adaptar pela documentação oficial.

Orçamento de aprendizagem

Você precisa de computador, internet e espaço para os arquivos. A prática também requer ferramentas de execução, como Python/Git, que vamos instalar; a extensão da IA não substitui essas ferramentas. O acesso ao assistente depende da conta e das condições atuais. VPS, domínio, e-mail de produção e cópias externas entram depois. Não são necessários para escrever a ideia nem para a primeira página local.

Monte uma pequena lista com item, finalidade, cobrança inicial, renovação, titular e data de revisão. Se não conhece um preço, marque “consultar” em seu planejamento privado e não contrate por impulso. Um preço mensal anunciado pode depender de pagamento antecipado por período maior. Compare total a pagar e renovação, não só o número maior do anúncio.

Oficina de orientação: abra uma pasta, crie `diario.md`, salve uma frase, localize o arquivo e mostre a pasta pelo terminal. **Resposta comentada:** o sucesso é reconhecer a mesma pasta no editor e em `pwd`; não há prêmio por fazer rápido. Essa orientação reduzirá erros durante todo o livro.

3. Oficina de início: a IA dentro do editor

Você já tem um lugar para guardar o trabalho. Agora vai colocar um assistente ao lado dos arquivos. O autor prefere a extensão no VS Code porque conversa, arquivos e revisão de mudanças ficam próximos. Vamos usar essa proximidade para aprender: você pede uma alteração pequena, encontra o arquivo e observa o resultado.

Escolha **Codex ou Claude Code**. Ambos são alternativas no método; não é necessário executar dois assistentes ao mesmo tempo. Duas ferramentas alterando os mesmos arquivos podem criar conflitos difíceis de entender. Uma troca de assistente será mais simples quando a documentação estiver salva.

Instalar a extensão que você realmente quer

No VS Code, abra Extensões com Ctrl+Shift+X ou Cmd+Shift+X no Mac. Pesquise o nome. Confira o publicador: OpenAI para Codex, Anthropic para Claude Code. Use a página oficial para chegar à listagem se houver extensões de nomes semelhantes. Leia a descrição e instale a extensão correta, evitando pacotes de prompts que apenas usam o nome da ferramenta.

No Windows com WSL, observe a janela em que trabalha. O editor pode distinguir instalação local de instalação no ambiente WSL. Siga a indicação da extensão para a janela atual. Se o painel não reconhecer arquivos Linux, primeiro confirme que abriu a pasta pela conexão WSL. Reinstalar tudo raramente é o primeiro diagnóstico adequado.

Depois abra o painel da ferramenta e conclua a autenticação pelo fluxo oficial no navegador. Verifique a conta antes de autorizar. Uma assinatura ou sessão de outro produto não deve ser

presumida como acesso ilimitado. Consulte os limites da sua conta. Não precisa criar uma chave de API só porque encontrou um tutorial que usa outro modo de autenticação.

O acesso à conta, o plano e a versão da extensão podem mudar. Esta edição consultou as páginas oficiais em 19/10/2026, mas não executou login em extensões nas três plataformas. Por isso, botões podem variar. O resultado a conferir é estável: painel aberto, conta correta e conversa vinculada à pasta certa.

Caminho Codex

Após instalar, abra o ícone Codex. Se não estiver visível, use a paleta e procure a ação **Codex: Open Codex Sidebar**. Entre pela opção oferecida para sua conta. Na primeira conversa, mantenha o trabalho no projeto local. Não é necessário delegar à nuvem para este exercício.

Peça que o assistente informe a pasta atual e leia apenas `ideia.md`. Confira se o resumo corresponde ao arquivo que você escreveu. Antes de permitir execução, entenda o alcance das permissões mostradas. A documentação oficial descreve o uso da extensão e a revisão de alterações no editor; esta oficina organiza uma primeira tarefa didática, não reproduz uma sessão real já executada pelo leitor. Extensão Codex.

Caminho Claude Code

Após instalar a extensão da Anthropic, abra o painel pelo ícone ou pela paleta procurando Claude Code. Conclua o login. A extensão inclui os componentes necessários ao painel de conversa; instalar separadamente o comando `claude` é uma escolha adicional para usar a interface de terminal, não o caminho principal desta oficina.

Use o modo de planejamento quando quiser análise antes de alterações e confira o modo realmente selecionado. Os controles de permissão e aceitação podem variar por versão. Comece pedindo leitura do arquivo de ideia, como no caminho Codex. O importante é separar “compreender” de “alterar”. Claude Code no VS Code.

Prepare o arquivo que será lido

No explorador do VS Code, clique em Novo arquivo na raiz da pasta e salve como `ideia.md`. Escreva: “Quero estudar um sistema de ordens de serviço fictício. Um gerente cria e acompanha; cada técnico vê apenas o próprio trabalho. Ainda não quero banco nem publicação nesta oficina”. Confira o nome e salve. O capítulo de requisitos aprofundará esse texto; para o primeiro teste de leitura, ele já fornece contexto suficiente.

A primeira mensagem não pede um sistema inteiro

Cole no chat da extensão, com a pasta do exercício aberta:

```
Estou aprendendo. Primeiro diga qual pasta do projeto está
aberta.
Leia somente ideia.md e resuma o objetivo em linguagem simples.
Não altere arquivos, não instale nada e não publique nada
agora.
Liste o que está claro e o que precisa de decisão humana.
Explique qualquer termo técnico que precisar usar.
```

A resposta deve citar sua ideia, não inventar um produto. Se a IA diz que não consegue ler o arquivo, confira o nome e a pasta. Se diz “não encontrei”, talvez você ainda não tenha salvo. Se resume outro projeto, pare e corrija o contexto antes de permitir edição. Esse teste custa menos do que descobrir o engano depois de vinte arquivos alterados.

Depois peça uma ação reversível:

```
Crie docs/resumo-inicial.md com o resumo confirmado e uma seção
Dúvidas pendentes. Não acrescente funções ao escopo.
Ao terminar, informe o caminho criado e explique como abri-lo.
```

Abra o arquivo pelo explorador. Compare com a resposta. A ferramenta realmente escreveu? A mudança ficou dentro da pasta? O arquivo contém aquilo que você autorizou? Você acabou de realizar o ciclo básico: pedido, ação, observação e conferência.

Seu primeiro resultado visual, sem servidor contratado

Para perder o medo do código, faça uma página isolada antes da aplicação com banco. Essa página é um exercício, não a tela final do sistema. Peça:

```
Crie uma pasta aquecimento com um único arquivo index.html.
Faça uma página em português com o título Oficina Clara,
um parágrafo explicando que este é um exercício fictício e
três cartões: ordens abertas, em andamento e concluídas.
Use números fictícios identificados como exemplo.
Use HTML e CSS no próprio arquivo, sem bibliotecas,
rastreamento,
requisições externas, formulário ou coleta de dados.
Explique como abrir o arquivo no navegador local e o que ele
não faz.
```

No gerenciador de arquivos, entre em `aquecimento` e abra `index.html` com seu navegador. O endereço começará normalmente com `file:`. Não procure esse caminho num mecanismo de busca: abra o arquivo. Você verá uma página estática. Ela não salva ordens, não autentica usuários e não representa um sistema pronto. Essa limitação é parte do exercício.

Peça para mudar o título para “Meu painel de estudo”. Atualize a aba do navegador. Se nada mudar, confira se salvou o arquivo, se abriu a mesma cópia e se o navegador atualizou. Você não precisa abrir uma VPS para ver texto e cor mudarem numa página local.

Ler uma mudança sem saber tudo de programação

O editor pode mostrar diferenças, chamadas **diff**. Uma área removida e outra acrescentada indicam o que mudou; as cores não substituem a leitura. Se você pediu apenas o título e aparecerem instalações, arquivos de rede ou coleta de e-mail, a tarefa cresceu além do necessário. Peça explicação e revise antes de continuar.

Você não precisa compreender cada símbolo na primeira sessão. Precisa reconhecer a relação entre pedido e resultado. “O título mudou e nenhum outro comportamento foi solicitado” já é uma conferência útil. Para regras de acesso e banco, a revisão será mais rigorosa e incluirá testes, como veremos depois.

Quando aparece um pedido de permissão

Permissão significa que a ferramenta quer executar uma ação com determinado alcance. Leia o que será feito, onde e por quê. Uma instalação muda seu ambiente. Um comando de exclusão pode remover trabalho. Uma chamada externa pode transmitir dados. Não use “aceitar tudo” para substituir compreensão.

Você pode pedir: “Explique o efeito deste comando antes de executá-lo; indique quais arquivos ou serviços serão alterados e como verificar”. Se não consegue entender a consequência, reduza a tarefa. Para a primeira oficina, escrita na pasta do exercício basta. Não há razão para dar acesso a documentos pessoais, bancos de clientes ou credenciais.

Se o painel não funcionar

Sinal	Primeira verificação
Não encontro o ícone	Extensão correta instalada e habilitada; procurar ação na paleta
Login abre e não conclui	Conta, navegador e mensagem do retorno; não repetir criação de contas

Sinal	Primeira verificação
IA não vê arquivo	Pasta aberta, arquivo salvo e ambiente local/WSL correto
Ferramenta pede cobrança	Conferir plano e modalidade; não autorizar uso pago sem entender
Resposta parou por limite	Salvar progresso e retomar depois; não perder decisões no chat
Comando não existe	Identificar se é ferramenta do sistema ou da extensão; instalar só o necessário

Saída desta oficina: um resumo salvo e uma página estática que você abriu e modificou. **Pergunta de compreensão:** por que essa página ainda não é o sistema de Clara? **Resposta:** ela apenas apresenta informações fixas; não autentica, aplica regras nem grava ordens. No próximo passo construiremos as instruções que permitirão à IA produzir essas partes.

4. O que você realmente entrega

Um sistema sob encomenda resolve um processo de um cliente identificado. Você recebe para descobrir, delimitar, implementar e entregar uma solução. O programa é uma parte do serviço: há também entendimento do problema, acesso, dados, documentação, responsabilidade e transição para quem vai operar.

No método profissional do autor, cada cliente recebe uma instalação exclusiva, paga sua hospedagem e pode contratar evoluções separadamente. Isso pede menos padronização de produto do que um serviço vendido a centenas de empresas, mas ainda exige proposta, negociação, limites e manutenção. Um sistema que serve a uma empresa pode depender de suporte mesmo sem mensalidade contratada com você.

O caso que vamos acompanhar

Na Oficina Clara, o gerente registra solicitações em mensagens. Dois técnicos perguntam quem deve atender cada cliente, e o gerente perde tempo reconstruindo o histórico. A primeira versão mostrará ordens de serviço com cliente, descrição, técnico responsável e estado. O gerente verá todas; cada técnico verá somente as suas.

Uma ordem passa de aberta para em andamento e depois concluída. Não haverá pagamento, nota fiscal, upload de fotos, aplicativo de celular, trabalho offline ou várias oficinas na mesma instalação. O laboratório lista as 100 ordens mais recentes visíveis; se o cliente precisar pesquisar anos de histórico, isso será outra entrega, com paginação e testes. Uma restrição explícita ajuda a escolher um projeto pequeno o suficiente para compreender.

O resultado do livro é um caminho de trabalho: necessidade → requisitos → documentação → implementação → testes → homologação → entrega e operação. Há retorno entre as etapas. Um teste pode revelar requisito ambíguo; uma limitação de infraestrutura pode exigir simplificação.

Quatro ambientes diferentes

Desenvolvimento é seu computador com dados inventados. Você pode experimentar e refazer. **Teste automatizado** cria dados temporários e confere regras repetidamente. **Homologação** se parece com produção, mas é usada para validação e ensaios sem expor operações reais. **Produção** é o ambiente do cliente, com usuários e consequências reais.

Salvar o código no GitHub não instala o sistema. Abrir a página inicial não comprova permissões. Fazer backup não comprova restauração. Cada uma dessas ações terá uma conferência própria.

A responsabilidade continua humana

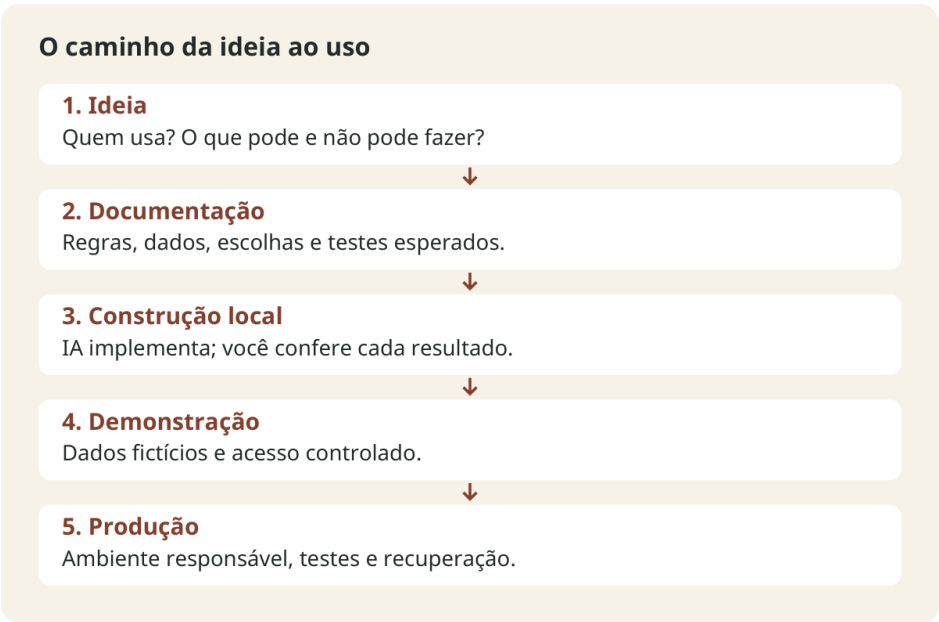
A IA pode sugerir arquitetura, escrever código, explicar erros e propor testes. Você precisa reconhecer o que mudou, executar verificações e saber quando buscar ajuda. Antes de assumir um cliente, consiga responder: onde os dados ficam, quem pode acessá-los, como recuperar o serviço e quem recebe o chamado quando ele falha.

O autor relata velocidade em projetos particulares. Um iniciante ainda precisa aprender vocabulário, ferramentas e diagnóstico. Para estimar prazo, divida o serviço em descoberta, implementação, testes, implantação, validação e correções. Registre horas efetivas; não venda um prazo antes de observar sua capacidade em projetos pequenos.

Sua primeira atividade

Escolha um processo simples que conhece. Descreva quem o executa, qual problema ocorre, o custo da dificuldade e o que indicaria melhora. Compare com planilha ou ferramenta já existente. Se uma solução pronta atender bem, indique-a; o interesse do cliente não depende de haver código exclusivo.

Conferência: você deve conseguir explicar em um minuto a necessidade da Oficina Clara e citar três coisas fora do escopo. Se sua explicação for “um sistema com tudo”, volte ao problema. Essa descrição orientará os requisitos da construção e, mais adiante, a conversa comercial com um possível cliente.



O método do autor, explicado sem pular a parte invisível

Warley usa esse método profissionalmente: entende a necessidade, escreve o que o sistema deve e não deve fazer, aprofunda a documentação com IA, escolhe tecnologias, desenvolve por fases e prepara a implantação. Ele relata já ter obtido receita com sistemas feitos dessa forma. Essa é a origem prática do processo ensinado.

O livro transforma essa prática em um percurso de aprendizagem. O exemplo fictício não representa um cliente do autor. Também não há prazo padrão para um iniciante: você estimará o trabalho a partir do escopo e das horas que realmente observar nos exercícios. A velocidade cresce com repertório, reaproveitamento autorizado e capacidade de diagnosticar problemas.

Vamos preservar a sequência, acrescentando explicações para quem não sabe o que uma etapa esconde. Uma descrição de negócio produz um documento legível. Um documento legível orienta a escolha das peças. As peças precisam de uma ordem de construção. A construção precisa produzir evidências. Por fim, o sistema precisa continuar utilizável quando você fechar o computador.

Seu mapa de viagem

Passagem	Pergunta que você responde	Evidência para guardar
Ideia	Qual dificuldade precisa diminuir?	Uma descrição com situação concreta
Pré-documentação	Quem faz o quê e o que não pode acontecer?	Regras e dúvidas separadas
Stack e segurança	Quais peças resolvem isso com custo administrável?	Decisões com motivos e alternativas
Documentação completa	Como construir, verificar e operar?	Requisitos, arquitetura, fases, testes e operação
Desenvolvimento	A próxima parte funciona e respeita as regras?	Arquivos, teste e histórico
Demonstração	O responsável reconhece o processo real?	Roteiro preenchido e correções

Passagem	Pergunta que você responde	Evidência para guardar
Produção	Dados, acesso e recuperação estão preparados?	Aceite técnico e de negócio, com responsáveis
Continuidade	Quem atualiza e responde quando falha?	Passagem de operação e acordo de suporte

Uma passagem pode revelar algo que exige voltar. Se o cliente diz que técnicos precisam trabalhar sem internet, a restrição “online” muda. Não resolva isso pedindo à IA “ative offline” sem avaliar dados, sincronização, conflitos e custos. Reabrir uma decisão é mais barato do que construir sobre uma suposição errada.

O que significa sistema completo neste percurso

Completo é relativo a um escopo. Nosso laboratório tem um processo inteiro, pequeno e observável: criar, atribuir, acompanhar e concluir uma ordem. Ele não é completo para administrar uma empresa inteira. Um projeto comercial pode exigir anexos, recuperação mais forte, limitação de abuso, monitoramento e integrações. Cada ampliação recebe requisitos e testes próprios.

Você encontrará dois caminhos complementares. A **oficina de criação** ensina a começar numa pasta vazia e dirigir a IA. O **projeto de referência** acompanha os materiais e permite observar uma implementação conhecida. Não compare arquivos linha por linha como se existisse uma única solução correta; compare comportamentos, permissões e critérios de aceite. Sua IA pode produzir outra organização de código.

Antes de prosseguir, explique para outra pessoa: “Vou construir uma versão pequena que faz isto, não aquilo, e vou conferir assim”. Se a pessoa perguntar por uma função excluída, a pergunta revelou algo útil. Uma boa explicação consegue dizer não sem esconder o que falta.